StratumRO Memory Architecture Report

Date: 21 October 2026. Scope: how to give every AI model and agent that works on StratumRO one shared, trustworthy memory, whether to use Obsidian or Git, what "app" the agents connect to, and what my role as researcher is.

Evidence labels: [REPO] read in your repository, [PUB] from a public source with URL, [CALC] computed from numbers shown, [EST] my engineering judgement, [UNVERIFIED] not confirmed.

1. Direct answer

1. Obsidian and Git are not alternatives. Use both on the same folder. A private Git repository of Markdown files is the source of truth (history, branches, review, two machines). Obsidian is the window you use to read, link and edit those same files. [EST]
2. The "app that every agent connects to" already exists as a standard: an MCP server pointed at that folder. Do not build a custom app first. Start with agents reading the folder directly, add an existing MCP memory server in stage 2, and build your own thin one only if tests show a gap. [EST]
3. The most important part is not the software. It is the rule about who may write what. Agents write proposals to an `inbox/` folder on their own branch; only you promote a note to `canon/`. This is what stops a wrong guess from one model becoming a "fact" every other model trusts. [EST, supported by the poisoning research in section 5]
4. Keep the always-loaded instructions small. The only controlled study I found on repository context files shows that large or machine-written ones cost more and do not help (section 5). [PUB]
5. My role (researcher) is to produce sourced notes with URLs, dates, evidence labels and a re-check date, which you drop into `inbox/`. I cannot write into your vault today (section 7).

I built a starter vault with the folders, rules, templates and seven seed notes taken from the work already done in this project: `stratumro-brain-starter.zip`.

2. What "memory" means here

Different kinds of memory need different homes. Putting all of it in one "memory app" is how projects get confused, and your repo already shows the symptom: 3,231 tracked files, 18 agent definitions and many overlapping status documents [REPO, Rebuild Blueprint].

Kind	Example	Home	Who writes	Loaded when
Rules	"Run tests with this command; never edit `canon/`"	`AGENTS.md` in the code repo (minimal)	You	Always

Kind	Example	Home	Who writes	Loaded when
Project state	Current phase, gate status	One file, `docs/STATE.md`	You or an agent, reviewed	Start of task
Decisions	"SNAP only after the QGIS phase"	`canon/decisions/`	You (approve)	By index
Verified facts	Law 169/2026 in force 25 Aug 2026	`canon/facts/`	Researcher proposes, you approve	By index
Lessons	"Recall stayed at 6.15% after tuning"	`canon/lessons/`	Agents propose, you approve	By index
Task cards and handoffs	One task, one acceptance test	`tasks/` in code repo; handoff notes in `inbox/`	Agents	Per task
Agent scratch	Session logs, trials	`agents/<name>/`	That agent only	Never by others
Run provenance	Input hash, parameters, outputs	Run folders in the code repo	The engine	On audit
Vendor built-in memories	Each product's own saved memory	Inside each product	The product	Automatically

Vendor built-in memories are separate stores that you cannot inspect uniformly. I did not research them. Treat them as caches that may be wrong, and keep the real record in your files. [EST]

3. Recommended design

See the diagram `fig10_memory_architecture.png`.

Components:

1. Code repository (existing). Gets three lines in `AGENTS.md` pointing to the vault. Both OpenCode and Antigravity read `AGENTS.md`: OpenCode reads it first and falls back to `CLAUDE.md` (OpenCode rules); Antigravity combines `AGENTS.md`, `GEMINI.md` and `.agents/rules` (Antigravity rules). One pointer therefore reaches both. [PUB]
2. Memory repository `stratumro-brain` (new, private Git). Folders: `canon/` (approved), `inbox/` (untrusted proposals), `agents/<name>/` (private scratch), `archive/`. A one-screen `INDEX.md` is the entry point.
3. Obsidian opens the same folder on both machines. It gives you backlinks and a graph view of the same Markdown files. [EST]
4. Connector layer. Stage 1: agents read files directly. Stage 2: an MCP memory server over the folder for search. Both are described in section 6.
5. Search index. If you add semantic search, treat the index as a rebuildable cache. If it is lost, the Markdown still holds everything. [EST]
6. Review gate. A weekly 30-minute pass over the `inbox/` diff, where you promote, reject or archive each note.

Why Git is the base: every change has an author, a date and a diff; each agent works on its own branch, so two agents never overwrite each other; you can revert a bad note; and the same repository serves the workstation and the laptop. [EST] A comparable design is used by Letta Code, which stores agent memory in git-backed "context repositories" and uses git worktrees so concurrent subagents do not collide (Letta blog). [PUB]

Why Obsidian is only the editor: it works on a plain folder of Markdown, so it does not lock the memory into one program. Basic Memory, for example, documents that its files are ordinary Markdown that Obsidian can open and edit (Basic Memory). [PUB]

4. Options compared

Option	What it is	Fit for StratumRO	Notes
Git repo of Markdown, read directly	Files plus `AGENTS.md` pointer	Best first step	No new software; reviewable. Search is only by file name and text. [EST]
Obsidian alone (no Git)	Local vault	Weak as the only layer	No review gate, no branches; two machines need a sync tool. [EST]
Basic Memory	Markdown source of truth, MCP-native, wikilinks as graph; Obsidian can open the folder	Strong stage-2 candidate	Local use is free under AGPL-3.0; a paid cloud exists, listed at $15 per seat per month by a third party (Coworker). Clients named in its README include Claude Desktop, Claude Code, Codex, Cursor and VS Code. Antigravity and OpenCode are not named, so test them. [PUB]
Obsidian Local REST API (with MCP endpoint)	Plugin that exposes the vault to tools	Good if you want to keep editing in Obsidian	Needs Obsidian running and an API key (plugin). [PUB]
Official MCP Memory server	Knowledge graph stored in a JSONL file; 9 tools	Fine for a trial, not for the base	Its own README calls the repository servers reference implementations, not production-ready (README). Data is a graph, not readable notes. [PUB]
mcp-memory-service	Self-hosted memory service: MCP, REST, OAuth, local sqlite-vec search, `X-Agent-ID` tagging per agent	Candidate if you want one shared semantic store	Apache-2.0; project documentation claims 5 ms retrieval and support for 14+ clients (GitHub). I read the README and package metadata only, not the code. Stores memories in its own database, not as Markdown, so you lose the readable-files property unless you export. [PUB, UNVERIFIED performance]
Mem0 (hosted MCP)	Cloud memory with an MCP endpoint	Not recommended for your data	Memories live in Mem0's cloud (docs). A third party reports that the local OpenMemory option was removed on 29 July 2026 (Coworker); I could not confirm this on Mem0's own pages. [PUB]
Letta Code	Agent runtime with git-backed memory	Interesting but replaces your harness	Shared memory repositories require cloud-hosted agents (Letta docs). You would be switching tools, which your own project history argues against. [PUB, EST]
Zep / Graphiti	Temporal knowledge graph	Overbuilt	Needs a graph database (Neo4j or FalkorDB). [PUB]

Benchmark caution: vendors publish memory recall figures of 80 to 93%, but those are not comparable across vendors (Developers Digest). One independent test, run once by one author on a single LoCoMo conversation (419 turns, 120 questions), measured recall of 57.6% (GoodMem), 52.2% (Letta), 50.0% (Mem0), 45.7% (LangMem) and 37.0% (Zep/Graphiti) (Shabbir). That is one small run and says nothing about geospatial notes, so the right conclusion is: test any candidate on your own questions (stage 2). [PUB, EST]

5. What the evidence says

Always-loaded context should be small. A study of coding-agent context files found that LLM-generated files lowered task success by about 3% on average and raised cost by more than 20%. Developer-written files improved success by about 4% on average and raised cost by up to 19%. The authors recommend describing only minimal requirements (arXiv 2602.11988). A later study suggests the quality of the guidance is the real variable, not the file itself (arXiv 2606.20512). [PUB] Consequence: `AGENTS.md` carries rules and a pointer; facts and history live in the vault and are opened on demand through `INDEX.md`. [EST]

Shared memory is an attack and error surface. In the MINJA research (NeurIPS 2025), an attacker who could only interact with an agent injected malicious memories with over 95% success and caused attacks with over 70% success, under idealized laboratory conditions (Schneider, Value Add VC summary). Defences listed by these sources include provenance tagging, quarantine of new memories, trust-aware retrieval, write validation and human confirmation (WorkOS). [PUB] Your threat is mostly accidental: an agent writes a confident wrong claim (for example a repealed law as current, which `pug_product.py` already does [REPO]) and every later agent repeats it. The same defences apply. [EST]

Applied to the design: provenance is the frontmatter (author, source, date); quarantine is `inbox/`; trust partition is `canon/` versus everything else; human confirmation is your weekly merge.

6. Rollout in three stages

Stage 1 (about one hour, no new software)

1. Create the private repository from the starter zip and clone it on both machines.
2. Open it in Obsidian.
3. Add the pointer lines from `MEMORY_RULES.md` to the code repo's `AGENTS.md`.
4. Pass check: give each of your tools (Antigravity, OpenCode, Claude, Gemini) the same five project questions, for example "which phase comes before SNAP?" and "what law is in force for PUG?". Each should answer from `canon/` and cite the note id. If not, fix `INDEX.md`, not the tool.

Stage 2 (when stage 1 passes; about one evening)

1. Add one MCP memory server over the folder. Candidates: Basic Memory, or Obsidian Local REST API, or the plain filesystem MCP server on the vault path. Install commands for Basic Memory are in its README: `claude mcp add basic-memory -- uvx --prerelease=allow basic-memory mcp`. [PUB]

2. Connect it to OpenCode and Antigravity through their MCP settings; I did not verify the exact settings for either, so follow their current documentation. [UNVERIFIED]
3. Pass check: write about 20 real project questions with known answers (from `canon/`). Score each setup: correct answer, correct note cited, tokens used. Keep the setup only if it beats stage 1 on answers or tokens. [EST]

Stage 3 (only if stage 2 shows a gap)
A small custom MCP server with about six tools: `read_index`, `search`, `read_note`, `propose_note` (writes to `inbox/` on a branch), `list_stale`, `promote` (owner only). It is justified only when you need the promotion rules enforced by code rather than by convention. [EST]

7. My role as researcher

What I can do today. I research questions you or an agent cannot settle from the repo alone (law, tool documentation, model comparisons, tool versions) and return a note in the vault format. A research note must contain:

1. Question, in one sentence.
2. Answer, three to six sentences.
3. Sources: URL and retrieved date for every claim; the specific page, not a home page.
4. Evidence labels on every claim.
5. What I could not verify.
6. Confidence and a `recheck_by` date.

The frontier models then consume the short answer, not the raw search. That is the correct split: I handle breadth and sourcing, they handle code and reasoning over the repository. The notes `F-001` and `F-002` in the starter vault show the format.

What I cannot do today. I have no connection to your vault or Git hosting in this session. A GitHub connector exists on the platform but is not connected. Until you connect it, you copy my notes into `inbox/`. If you choose to connect it, I would write only to a branch or pull request in `stratumro-brain`, never to `canon/`; that choice is yours, and I would not need access to the code repository for this. [EST]

I also cannot hold a live link to your local MCP server from here, because it runs on your machine. The note-in, note-out hand-off through Git is the safe equivalent. [EST]

Limit on my own memory. My knowledge of your project comes from this conversation and the repository snapshot I read (HEAD 64a5cec, 20 October 2026, shallow clone) [REPO]. Anything newer is unknown to me until it is in the vault.

8. Two machines

1. Both machines clone the same repository and pull before work. Commit and push at the end of a session.

```
1. Agents work on `agent/<name>/<task>` branches, so the workstation and the laptop r
```

1. Do not sync the vault through a file-sync tool on top of Git; two sync mechanisms on one folder cause conflicts. [EST]
2. Keep `.obsidian/workspace*.json` out of Git (the starter `.gitignore` does this). Obsidian has a community Git plugin; I did not verify its current features, so check its page before relying on it. [UNVERIFIED]
3. Local semantic search indexes are per machine and rebuildable; do not commit them.
4. The laptop (RTX 4060, 6 GB VRAM) should not run a local embedding model alongside a local LLM; use text search first, or CPU embeddings. [EST]

9. Rules that stop drift

1. One owner for `canon/`: you.
2. Agents never delete; they deprecate and link the replacement.
3. Notes are short (under about 300 words) and have one idea each.
4. Every non-owner note expires: `recheck_by` is mandatory. Law and tool-version notes expire in three months.
5. Each task card links to the note ids it relies on, so a result can be traced to the facts it used.
6. Conflicts are reported, not silently resolved.
7. Never store credentials or client survey data in the vault.

10. Not verified

1. How Antigravity and OpenCode connect to Basic Memory or Obsidian-based MCP servers in their current versions.
2. mcp-memory-service beyond its README, package file and documentation pages; its speed and client-support claims are the project's own.
3. The Obsidian Git plugin's current behaviour.
4. Whether the OpenMemory removal reported by a third party is accurate.
5. Whether any benchmark figure transfers to Romanian geospatial notes; only your stage-2 test can show this.
6. Vendor built-in memory features (not researched).
7. Subscription limits for your AI tools.
8. The two recommendations D-003 and D-004 in the vault are proposals; they become decisions only when you approve them.